

Zeroing the Instrument

A primer on null-control batteries for software-based measurement

Christian S. Loh, Ph.D.

Southern Illinois University

ORCID: [0000-0002-6318-3890](https://orcid.org/0000-0002-6318-3890)

Yanyan Sheng, Ph.D.

The University of Chicago

ORCID: [0000-0003-4871-2202](https://orcid.org/0000-0003-4871-2202)

September 19, 2026

Central question

What does the complete action-to-report pipeline produce when the target human mechanism is absent?

Null controls · measurement validation · research software · trace data · AI-assisted development

A Praxichnology Teaching Resource

github.com/praxichnology/primers

doi.org/10.5281/zenodo.20371003

Contents

1. Introduction	3
2. A Working System Can Still Mislead	6
3. Why the Null Response May Not Be Zero	9
4. Four Families of Checks	14
5. Building and Reporting a Battery	16
6. Limits and Responsibility	18
Availability and Status of Instructional Materials	19
Acknowledgments of AI Use	19
References	19
Appendix A. Companion Casebook	20
A.1 Guided lessons	21
A.2 Worked answers	24

1. Introduction

Software can be both the environment in which activity occurs and the instrument used to measure it. Learning platforms, serious games, simulations, and other research applications routinely turn interaction traces into evidence that a learner improved, a trainee performed competently, or an expert pattern emerged.

These systems can pass their software tests and operate exactly as designed while leaving a prior question unanswered:

What if the signal is coming from the software, not the user?

A working application can record, filter, group, and analyze its own data correctly while contributing structure that appears to come from the actor. Repeating the analysis will not necessarily expose that contribution. It may simply make a plausible and publishable result look more trustworthy.

This report introduces a **null-control battery** as an instrument-checking step for software whose own records support claims about actors. Its organizing idea is **zeroing**: determine the instrument's expected response under a stated no-mechanism condition. Zeroing does not mean forcing the reading to zero, and it does not mean subtracting an expected value from human observations. For a measure computed from a system's own records, the null response need not be zero.

We develop that argument through two hypothetical examples: the fictional game *Garden Defense* and a two-toss coin demonstration. Together they show how software architecture, grouping rules, outcome definitions, and analyst choices can shape a reported result even when the target actor-level mechanism is absent. The report then organizes the available checks into three null-control families and one analyst-choice sensitivity family, before giving a practical register for building and reporting a battery.

The [companion casebook in Appendix A](#) turns those examples into an inspectable instructional package: machine-readable synthetic inputs, guided exercises, worked answers, a completed battery register, and a dependency-free verification script. The casebook contains no human participants, research data, or empirical evidence. It reproduces declared examples; it does not validate a real game or instrument.

1.1 The central risk

Before trusting a laboratory assay, we can run a blank containing none of the substance being measured. In cooking, to know whether salt is responsible for the taste of a dish, we make the dish once without salt. Even a digital thermometer can be checked for accuracy by placing its probe in ice water to see whether it reads $\sim 0^{\circ}\text{C}$. Each check begins with a condition specified independently of what the instrument or procedure returns. We then observe the response under that condition.

Software-based instruments need the same kind of check. A learning platform may report that students who engaged early performed better later. A serious game may report that players who received delayed feedback showed higher competency. A clinical simulator may report that trainees who passed one checkpoint went on to better performance. All three claims are about people, but all three are computed from records the software itself produced.

More precisely, could the system have produced the reading even when the users did not possess the disposition, learning, or skill being inferred? This is possible because software does not merely collect a neutral record. Designers decide what counts as a learning event, how it is logged, where thresholds are placed, and how observations are aggregated as "scores." ***The consequence is easy to miss: those decisions can give the record a structure of its own.*** An analysis may then attribute to actors a pattern produced jointly by their actions, the environment, and the measuring procedure. The problem here is not the familiar gap between observing what users did and explaining why they did it. It is prior: **what does the complete action-to-report pipeline produce when the actor-level mechanism being inferred is absent?**

Why should software designers and researchers pay attention? Because software can run as designed while producing records or patterns that do not mean what its developers think they mean. This risk is documented in both education and professional software development. Kovanović et al. (2015) found that different methods of estimating students' time-on-task from learning-platform records changed study results; they also noted that estimation procedures and their consequences were often insufficiently examined. Drawing on thousands of Microsoft experiments, Dmitriev et al. (2017) documented recurring errors in interpreting recorded metrics. In one example, opening links in a new browser tab improved click tracking because the original page stayed open long enough to send more tracking events. A change in recorded clicks could therefore reflect the recording process, not simply a change in user activity.

For developers of serious games and other research software, the implication is that programming expertise alone does not establish that the resulting data support the intended interpretation. Measurement choices need explicit examination, even when the application works and its calculations are correct.

A **null-control battery** makes the instrument vulnerable to specific failures before its traces are used to support claims about people. Each control removes or breaks one possible source of the reported pattern. The battery then asks whether the instrument behaves as expected under that condition. It does not certify the instrument as valid in every respect, but it can provide evidence against the particular alternative explanations it was built to test.

1.2 Scope and audience

The scope is deliberately narrow:

This report concerns instrumented applications whose own interaction traces are used as evidence about actors.

Here, an **actor** is the person or automated agent whose actions produce the records. A **construct** is the skill, disposition, learning, or other attribute the software is intended to measure.

For example, in a serious game about emergency response, the players are the actors and procedural decision-making competence might be the construct. In an online learning platform, the students are the actors and algebraic problem-solving skill might be the construct.

It is not a claim about all technology, all measurement, or all research. Records used only to operate or optimize a service—for billing, uptime, crash reporting, engagement, retention, balance, or monetization—are outside the scope because no inference about the user is being made. If the same records are used as evidence about skill, competence, disposition, or another actor attribute, that inferential use brings the system within scope. Measurement instruments that are not software also fall outside it; they have their own validation traditions.

The practical audience includes software designers and developers, researchers who commission or build research tools, and researchers who assemble software with AI assistance despite having limited training in software testing. The obligation does not depend on how the code was produced. Handwritten, copied, generated, and hybrid systems all need the executed program, settings, and data path to be inspectable when their records support claims about people.

1.3 From a research claim to a fitted control

Statistical inference is more than selecting a test and reporting its result. It is part of a reasoned case connecting an observation to a claim about actors. Abelson (1995) called this *principled argument*. Here, *principled* does not mean morally superior or derived from “first principles.” It means that each inferential choice follows an explicit reason that readers can inspect and challenge. A small *p*-value does not answer every methodological objection. A credible argument anticipates plausible alternative explanations and constructs appropriate comparisons to examine whether those explanations can account for the result (pp. 4–6, 12–13).

The reasoning begins before the calculation:

- 1 **State the claim.** What are the software records being used to say about an actor?
- 2 **Name the observation.** What recorded or computed pattern will be offered as evidence?
- 3 **State the warrant.** Why should that observation count as evidence for the claim? What must be true about the environment, records, measure, and comparison for that interpretation to hold?
- 4 **Name a plausible rival explanation.** How else could the same pattern have been produced if the proposed actor mechanism were absent?
- 5 **Fit a check to that rival.** Remove the proposed mechanism or break the relationship suspected of producing the pattern while preserving the parts of the path needed to test the objection.
- 6 **Judge the result against the warranted reference.** The expected comparison follows from how the check was constructed; it is not always zero.
- 7 **State a bounded conclusion.** Say which objection the check addressed, what happened, and what remains untested.

This report adapts that sequence to instrumented software. Each control is constructed for one named objection—noise, label assignment, software architecture, or analyst choice—and the conclusion is limited to that objection. The comparison condition may remove the target mechanism or deliberately break the connection suspected of producing the result. Its output must then be judged against the reference warranted by its construction.

Instrumented software adds a further opportunity: a specified no-mechanism actor can pass through the implemented application, producing actions, records, group assignments, and reported measures through the same pipeline used for human data. Confidence grows across the battery as each condition answers the objection it was designed to expose.

1.4 What null and control mean here

Readers may associate *null control* with a null hypothesis or a trial control group. The terms are not synonymous.

Null is not shorthand for the null hypothesis. A null hypothesis is a statistical claim about a population or data-generating process that is evaluated using observed data. A null control is a **condition that is run**, constructed so that the target mechanism is absent or the relationship needed to produce the claimed pattern has been broken. The two become related when the null condition is used to characterize what the reported statistic returns under that specified condition. The resulting reference distribution need not be centered at zero.

Control is not shorthand for a trial control group. A trial control group provides a comparator for estimating what happens to people under a chosen treatment or condition. A null control asks what the **measuring system** reports under such a condition. Like a blank well in a laboratory assay, it checks the apparatus and pipeline; it does not establish whether a treatment works.

Their functions differ, so neither automatically substitutes for the other.

1.5 What this report does not claim

Two limits should be clear from the outset.

- 1 **A null-control battery tests the machinery that produces a measurement under specified conditions.** It does not establish that a software product improves practice, that training transfers, or that one approach outperforms another. Those questions require designs fitted to them. Zeroing a thermometer does not tell us whether the patient has a fever.
- 2 **The underlying practice is not new.** Laboratory science has long used conditions in which the expected result is nothing. Epidemiology formalized a related practice as the **negative control**: a condition, exposure, or outcome selected because the hypothesized mechanism should not produce an effect there (Lipsitch et al., 2010).

A note on the name. Abelson (1995) supplies the argumentative logic developed above, not the phrase *null control*. The phrase predates this work, but its use is not standardized.^[1] Here it names a condition constructed so that the target mechanism is absent or the relationship needed to produce the claimed pattern has been broken. We use it to distinguish this broader

software-instrument condition from the specific negative-control exposure and outcome methods formalized in epidemiology (Lipsitch et al., 2010). An enacted software null may nevertheless have a non-zero expected reading; determining and justifying that reading is the zeroing problem developed below.

[1] Earlier experimental uses include Mensor et al. (1998), who used a titanium O-ring as “the null control,” and Chiang et al. (2010), who left defects untreated as a null control. These examples document prior use of the phrase; they neither identify a single origin nor establish a common definition.

An instrumented application makes one especially useful check possible. Researchers can send a specified no-mechanism actor through the system and observe what the complete pipeline reports. This does not replace statistical or observational negative controls. It adds a condition that can expose structure introduced upstream by the software itself.

Figure 1 summarizes this boundary. Instrument checking asks what the measuring system reports under a stated no-mechanism condition. Human inference is a separate research question that requires a design and comparison fitted to the claim about people.

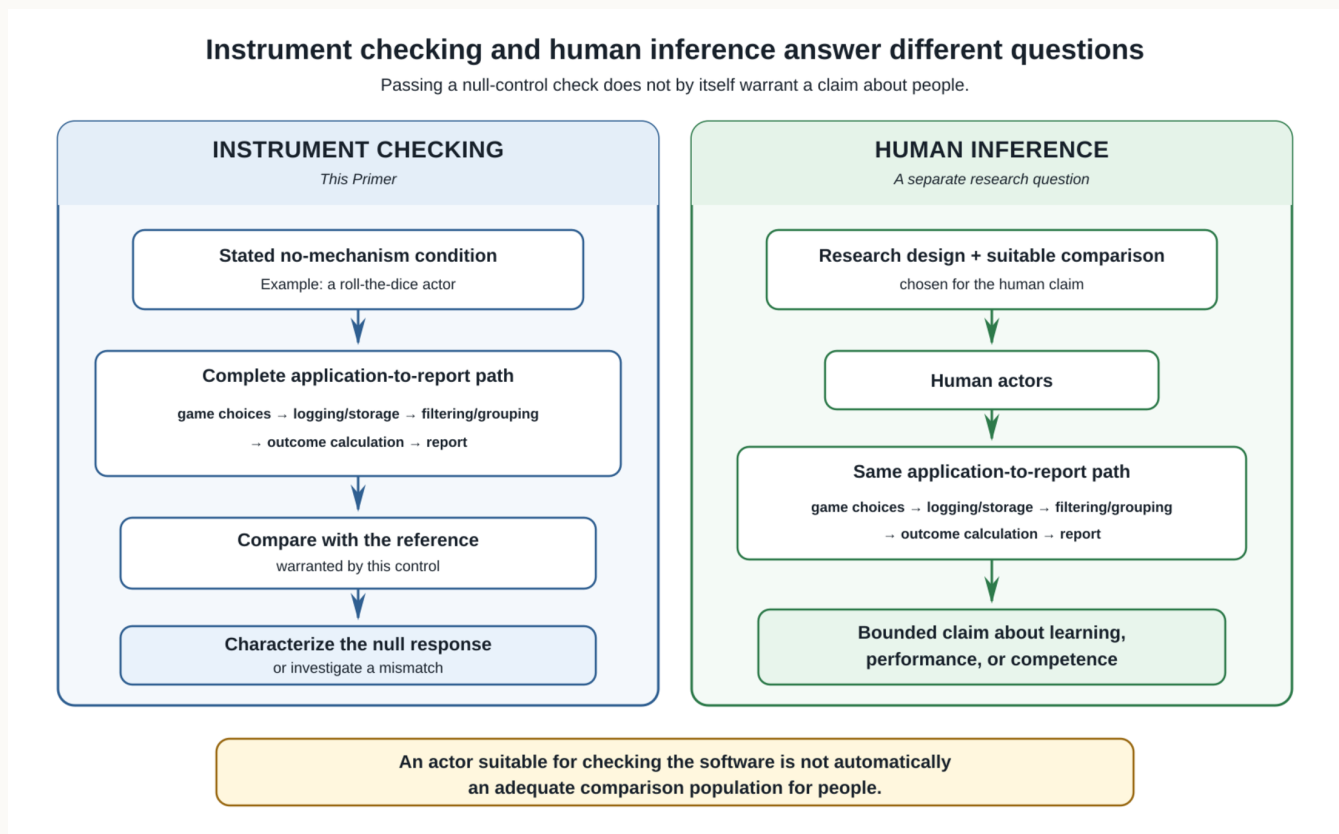


Figure 1. Instrument checking and human inference answer different questions. A null-control battery characterizes what the measuring system reports under stated conditions; it does not by itself validate the human construct or establish an actor-level effect.

2. A Working System Can Still Mislead

A null-control battery begins from a simple distinction: software can be correct as software and still be untested as a measuring instrument.

Jacobs and Wallach (2021) explain how an observable measure can diverge from the theoretical attribute it is intended to represent. They also argue that measurement modeling, which makes the connection between a construct and its observed measures explicit, has received little attention in computer science. A correctly computed performance score does not, by itself, establish learning. Null controls examine specific alternative explanations for a pattern; they do not establish construct validity on their own.

The following example moves from the most visible no-learning condition to the harder problem of choosing a control that can support a claim about people.

Note. The worked examples in Sections 2 and 3 are hypothetical and were constructed to illustrate the corresponding measurement problems. They are not research data or empirical evidence.

Consider a hypothetical worked example built around a fictional game called *Garden Defense*. It uses a familiar lane-based garden-defense pattern: players place defenses in several lanes while waves of pests approach a garden.^[2] Each completed round produces a score from 0 to 100. Players may replay the game, and the research team wants to study repeated performance.

[2] *Garden Defense* is fictional. Its general lane-based setup loosely resembles games such as *Plants vs. Zombies*; the scoring and later measurement rules were invented for this example.

2.1 Start with a visible no-learning actor

The simplest place to begin is to replace the player with an actor that cannot learn. At every decision point, it selects randomly. Because it remembers nothing and uses no feedback, any apparent learning pattern cannot reflect learning by the actor.

A roll-the-dice *Garden Defense* actor. Suppose 100 automated actors each complete three runs, producing 300 completed runs. At every decision point available to a human player, each actor selects one of the available options with equal probability. The game supplies the options; only the selection is random. One can imagine rolling a fair die whose faces represent the options currently available.

The actor does not remember earlier choices, respond to feedback, or plan ahead. Each run begins afresh. A later run may still score higher by chance, but the actor has not learned.

Run the no-learning condition through the implemented path. Send the automated actors through the same implemented path intended for the human data: the game presents the choices, records the selections, stores and filters the runs, applies the intended analysis, and produces the report. Replacing the choices only in a spreadsheet would test less of the instrument.

Repeating the control does not force the reading to 0. It reveals what the instrument ordinarily reports when learning is absent. Suppose, for example, that the learning index repeatedly returns a value near 0.0004 rather than 0.0000. That small value may come from finite runs or features of the game and pipeline. It is the instrument's null response under this condition, not evidence that the actor learned.

Researchers record this response rather than assume the instrument contributes nothing. A clear or recurring learning pattern means that the pipeline or measure requires investigation. The random actor does not model people; that question comes later.

A control tests only the stages it traverses. A generated table can test an estimator. An enacted actor can also test the game, logging, storage, filtering, analysis, and reporting that feed the estimator.

2.2 Audit the complete data path

Replacing the player with a null-control actor is only the first step. A reported result also depends on what the game records, what reaches storage, what the filters retain, what the analysis calculates, and what the report says:

actor → game choices → recorded actions → stored data → filtering → outcome → calculation → report

A correct calculation can still answer the wrong question if the wrong records reach it.

A correct calculation on the wrong records. One roll-the-dice *Garden Defense* actor completes three runs with scores of 52, 61, and 67. The correct mean of the three completed runs is 60. The final report instead gives 56.5.

The arithmetic is correct for 52 and 61. The problem occurred earlier: the third run was incorrectly marked incomplete, so the filter removed its score of 67 before the calculation. Whether the cause is an overlooked design step, a coding error, or AI-generated code that no one reviewed, the failure can remain silent: the game still appears to work while its data-capture and filtering chain drops a valid record. Nothing in the reported value of 56.5 reveals that a completed run is missing.

Put checkpoints along the path. The 100 actors in Section 2.1 should produce 300 completed run records. Check that all 300 leave the game, reach storage, survive the intended filters, and reach the analysis. The report should state how many records it actually used and identify any exclusions. Simple known cases should also return known answers: 52, 61, and 67 must produce a mean of 60.

Computational settings need checkpoints too. Suppose a publication analysis is set to run 10,000 resamples, while an interactive display uses 500 for speed. The final output must identify which path ran and record the number of resamples the estimator actually completed. An interval or null distribution may still look plausible when the requested setting never reached the estimator.

If a checkpoint fails, locate the missing or changed record—or the changed computational setting—before interpreting the result.

2.3 Check whether the assigning observation reappears

Sometimes every link in the chain works as intended, yet the measurement rules still create their own pattern. This can happen when the same observation helps assign the group and determine the outcome used to compare the groups.

Grouping and outcome overlap in *Garden Defense*. Suppose the research team does two things:

- 1 For illustration, they prespecify 65 points as the cutoff and bin players into two groups: **Early-High (Run 1 $\geq$ 65)** and **Early-Low (Run 1 $<$ 65)**.
- 2 They then compare the two groups on **Best Run**, defined as the highest score across all completed runs, including Run 1.

Both choices may sound reasonable. Early performance provides a convenient grouping rule, while Best Run records the highest level a player demonstrated. The difficulty appears only when the two rules are placed together: **Run 1 helps decide the group, and its score remains eligible to determine Best Run.**

For a constructed two-player illustration, suppose Player A scores 72, 55, and 56, while Player B scores 58, 55, and 56. Player A is Early-High with a Best Run of 72; Player B is Early-Low with a Best Run of 58. Their later records are identical, and both later-run maxima are 56. Their Best Runs nevertheless differ by 14 points because Run 1 serves two roles. This example shows what the rule can produce; 14 points is not an estimate of the expected null gap.

This does not make Best Run invalid. The first-run scores were performances those players demonstrated. The example shows that the grouping and outcome rules can contribute to the observed gap even when they operate exactly as designed.

Preserve the complete rule in the control. Repeat the same three-run no-learning actor design with fresh random choices. Each time, apply the same cutoff and the same Best Run rule, including Run 1, and record the group gap. These repeated controls show the group differences that ordinarily appear without learning. Compare an observed human gap with those no-learning gaps rather than automatically comparing it with zero or subtracting an average response.

Recomputing Best Run without Run 1 can help reveal the overlap, but it changes the outcome being measured. It is a diagnostic, not an automatic correction. Section 3 explains that distinction in more detail.

2.4 Separate a software check from a model of people

The roll-the-dice actor introduced in Section 2.1 is useful because it clearly cannot learn. But it is deliberately simple. People who do not learn during a study can still differ from one another in experience, caution, or preferred strategy. A uniform-random actor has no such stable differences.

Two no-learning populations in *Garden Defense*. The following numbers are invented for illustration. Suppose the uniform-random actors' Best Runs cluster between 55 and 65, and repeated groups of those actors usually differ by much less than 8 points. The illustrative human Best Runs, however, span 30 to 90 because the players have stable differences. A human group gap of 8 points therefore looks large when compared only with the uniform-random actors.

Now construct a second no-learning population. Give each actor different but fixed probabilities of selecting the available defenses. Those probabilities do not change across runs or respond to feedback. This creates stable play styles without learning. Both actor populations use the same number of runs, cutoff, and Best Run rule, but the second also produces Best Runs spanning 30 to 90. In this constructed illustration, these stable-style actors can produce group gaps around 8 points without learning.

The observed gap did not change. What changed was the no-learning population used to interpret it.

Check the control actor before trusting the comparison. Compare simple features such as the number of runs and the spread of Best Run scores. If the uniform-random actors vary much less than the people, keep them as a test of the pipeline but add a second no-learning actor that allows stable differences. If the answer changes, the first actor was useful for checking the software but was not enough to answer the question about people.

2.5 Combine the checks into a fitted battery

Taken together, the null conditions and their verification checks form a battery fitted to this instrument and claim:

- 1 an enacted no-learning control that traverses the implemented path;
- 2 pipeline checkpoints and known-answer checks that make the path inspectable;
- 3 a matched no-learning reference that preserves the grouping–outcome coupling; and
- 4 a population-fit check with a plausible heterogeneous no-mechanism foil.

These checks do not map one-to-one onto the four headings in Section 4. Sections 2.1 and 2.3 use enacted no-mechanism controls. Section 2.2 uses pipeline checkpoints and known-answer tests that can support any control family. Section 2.4 asks whether the control actor or population is adequate for the later claim about people. Each check supports only the objection it was built to address. For every control, researchers should state the actor or population it represents, the parts of the pipeline it traverses, the result it predicts, and the conclusion it is entitled to support.

The report therefore does not prescribe one battery for every instrument. It provides a reproducible method for constructing the battery an instrument needs. The battery should be **fitted to the instrument and the claim**. A control that is excellent for checking software behavior may be too simple for a claim about people.

These checks characterize what the instrument reports under stated no-mechanism conditions. The next section explains why that null response need not be zero and how it should—and should not—be used.

3. Why the Null Response May Not Be Zero

Before weighing flour, a baker places the empty bowl on a digital scale and presses *tare*—the scale's command for treating the bowl's current weight as zero. The later reading then represents the flour rather than the bowl. In a laboratory assay, a researcher runs a blank containing none of the substance being measured. Both checks ask the same question: *What does the instrument report when the thing of interest is absent?*

Software-derived measures need the same question, but they add a complication. The expected answer may not be the number zero. The game, grouping rule, outcome definition, and type of actor can together create a non-zero reading even when the target mechanism is absent.

That is why **zeroing** in this report means *finding the instrument's null response*, not forcing its output to zero.

3.1 A two-toss demonstration

The simplest demonstration does not require a game.

Imagine that each actor tosses a fair coin twice. Record Head as 1 and Tail as 0. Then apply two rules:

- 1 Label actors **First-Head** or **First-Tail** from the observed result of Toss 1.
- 2 Define the outcome as **Best Toss**, the higher value from the two tosses.

The coin is fair on every toss. Toss 2 is independent of Toss 1. The coin has no memory and cannot learn. Yet the expected group difference is not zero.

Every First-Head actor already has a 1, so that actor's Best Toss must be 1. A First-Tail actor begins with 0 and reaches a Best Toss of 1 only when Toss 2 is Head. That happens half the time. The expected Best Toss is therefore 1 for First-Head and .5 for First-Tail: an expected difference of .5.

Figure 2 lays out all four equally likely two-toss sequences. The asymmetry becomes visible once they are separated by the result of Toss 1.

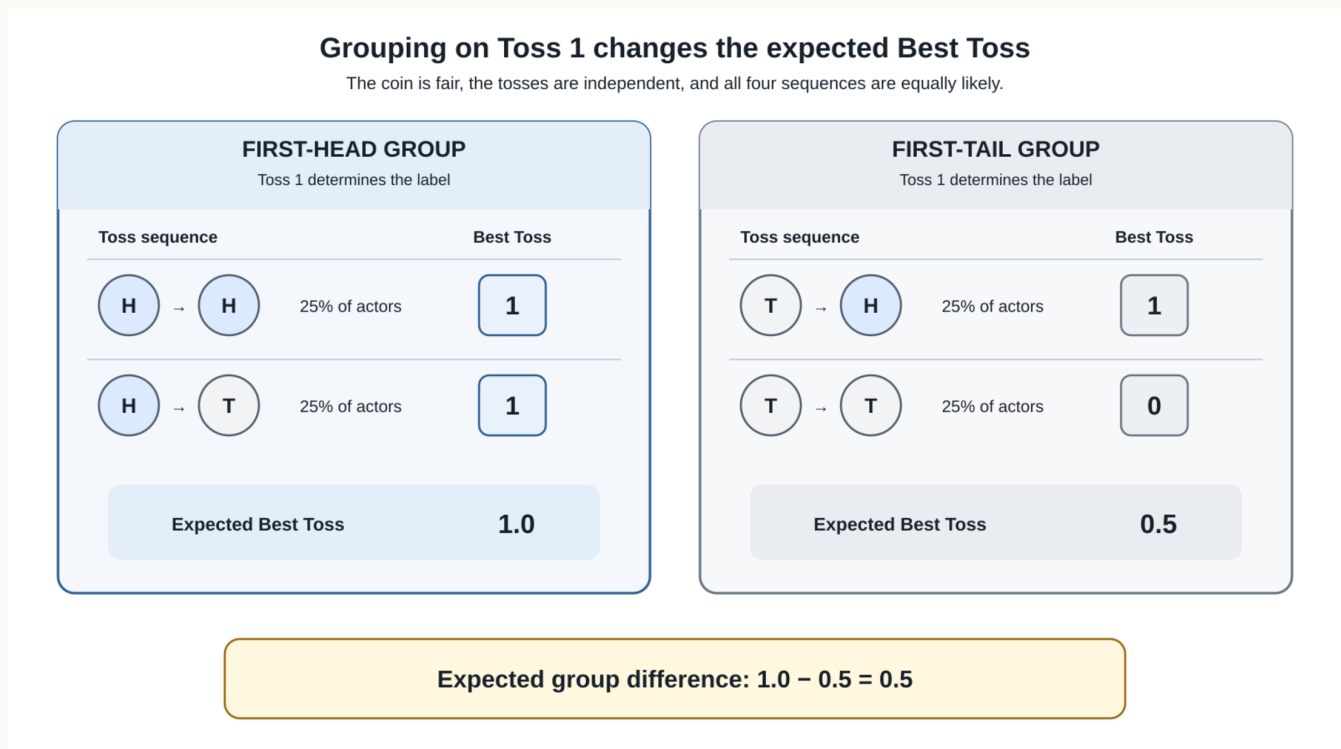


Figure 2. Grouping actors by Toss 1 produces a non-zero expected difference in Best Toss even though the coin is fair, the tosses are independent, and no learning is possible.

Nothing mysterious happened. Toss 1 was used twice: first to create the labels, and again as part of Best Toss. The First-Head group begins with an observation that already guarantees the maximum possible outcome.

With n total tosses, the expected difference is $2^{-(n-1)}$: .5 with two tosses, .25 with three, and so on. It remains positive for every finite n and approaches zero as additional tosses give First-Tail actors more chances to reach the same maximum. The explanation is entirely in the rules.

The fictional game has the same form. Replace Toss 1 with Run 1, First-Head/First-Tail with Early-High/Early-Low, and Best Toss with Best Run. If the run used to help create the group remains inside the outcome window, an independent no-learning actor can produce a non-zero group difference.

3.2 Two ways the assigning record can reappear

The general warning can now be stated without a formula:

When the same record helps form a group and also contributes to the outcome compared between groups, the instrument may have a non-zero response even when the target mechanism is absent.

The early record can reappear in two common ways.

Selection. The outcome selects a personal best, peak, or best-of- N . The assigning run remains eligible to be selected. This is the pattern in *Garden Defense* and the coin example.

Direct inclusion. The assigning run is added directly into a total, average, change score, or model. The direction of the resulting difference depends on the calculation. For example, if groups are formed from the first score and the outcome is *last minus first*, the higher-first group can show less gain even when later scores are independent. The null response can therefore be positive or negative. It is not a universal floor.

Figure 3 separates these two paths. In the first, the assigning record remains eligible to become the selected outcome. In the second, it enters the outcome calculation directly.

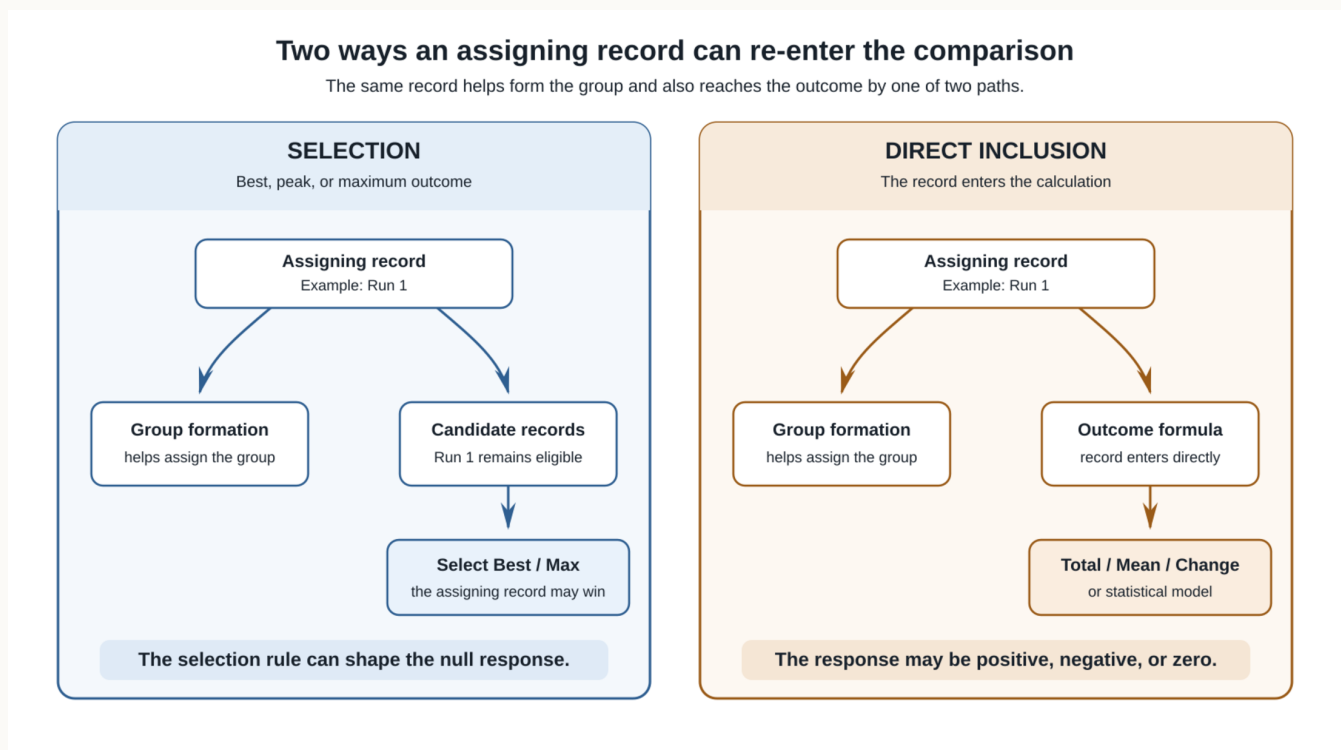


Figure 3. Two ways an assigning record can re-enter the comparison. It may remain eligible for selection as a best or maximum, or enter directly into a total, mean, change score, or model. The resulting null response depends on the rule and may be positive, negative, or zero.

This point is easy to misname. In this example, calling the positive gap residual regression to the mean would misidentify its source. For an independent random actor given the same number of later runs in each group, recomputing Best Run from those later runs only makes the expected group difference zero. The positive gap above arises because the assigning run remains inside the outcome.

That diagnostic is useful: compute the outcome once with the assigning run included and once without it. Under a control whose later-run distribution and run-count profile are independent of the Run 1 label, the difference should disappear in the second version. If it does not, another dependency remains to be explained.

3.3 Removing Run 1 changes the outcome

Removing Run 1 from Best Run breaks this particular coupling for the independent random actor. It may be a sensible redesign when the research question permits it. It is not a neutral repair.

Best Run usually means the best performance the actor demonstrated across the complete record. A player who performed best on the first attempt did demonstrate that capacity. Refusing to count it changes the meaning of the outcome merely to make one null actor produce zero.

The change can also affect other measures that reuse Best Run. Once the outcome window changes, any quantity derived from that outcome may change meaning as well.

As Section 2.4 showed, a no-learning population can retain stable differences across runs. Removing Run 1 therefore centers the result at zero only for a control whose later runs are independent of the first.

The apparent repair therefore answers a different question under one chosen actor model. Researchers should decide first what the measure is intended to mean and which no-mechanism population represents the objection. They should not redesign the construct simply to obtain a convenient zero.

3.4 Every null response needs an address

A statement such as "*the null response is .07*" is incomplete. The value belongs to a particular control. Give it an address by answering four questions:

- 1 **Who or what is the no-mechanism actor?** A uniform-random chooser, a fixed script, or a population with stable differences but no learning may answer different objections.
- 2 **What environment did the actor enter?** Thresholds, available choices, scoring rules, session boundaries, and logging behavior can all matter.
- 3 **How were groups and outcomes constructed?** A different cutoff, tie rule, or outcome window can change the response.
- 4 **How many observations did each actor contribute?** In a Best Run outcome, more runs provide more chances to reach a high score. A two-run population and a seven-run population need not share the same null response.

Change any part of that address and the ordinary response may change. The null response is not a constant belonging to Best Run, to a statistic, or to games in general. It belongs to the actor, environment, measurement rules, and run profile used to produce it.

The actor or population is therefore part of the address. As Section 2.4 showed, a uniform-random actor may characterize the software's response to uniform-random input without being an adequate comparison for a claim about people.

3.5 Locate the reading; do not subtract it

After repeating the matched control, keep the observed statistic unchanged and ask where it falls among the control readings.

Locate the observed reading in the distribution produced by the stated no-mechanism condition. Do not subtract the null expectation from the observed reading.

The control reading is not automatically a fixed amount of instrument bias. Grouping and Best Run are not simple additions, and the control does not show that the observed statistic equals a human effect plus the null response. Subtracting the two would create a new number without establishing what that number measures.

Figure 4 combines the two instructions: specify the control's address, then locate the unchanged observed reading in the resulting control distribution.

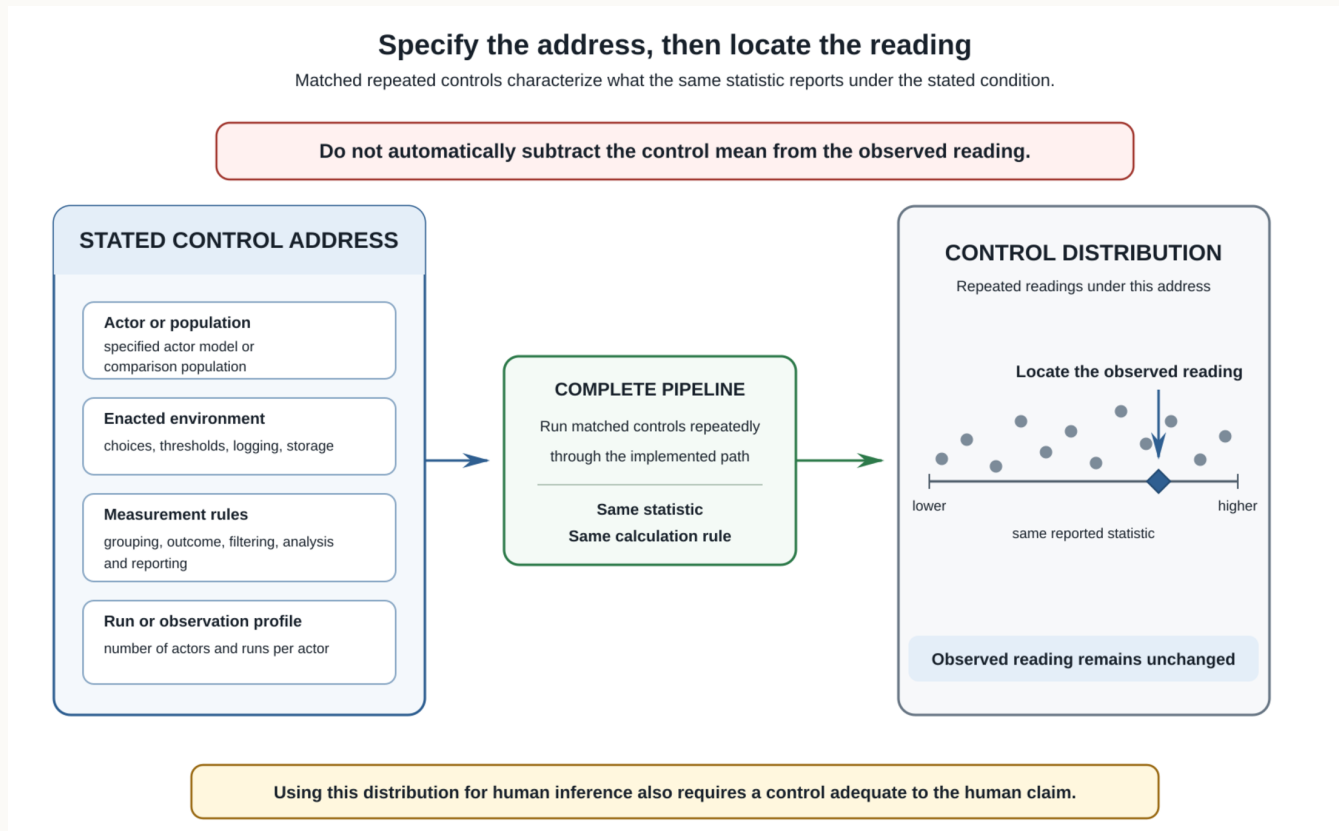


Figure 4. A null response has an address. The stated actor, environment, measurement rules, and run profile produce a control distribution. The observed reading is located in that distribution; the control mean is not automatically subtracted as an additive bias correction.

Consider a simple *Garden Defense* comparison in which the same signed group-gap statistic is calculated by the same rule every time. If repeated no-learning controls usually produce gaps from 6 to 9 points, an observed gap of 8 is ordinary under that condition. If they usually produce gaps from 0 to 2 points, the same observed gap of 8 is not reproduced by that condition. The observed statistic did not change; only its interpretation against the stated control changed.

Section 4 widens the view from the enacted controls used in these examples to other recurring objections. Section 5 then turns the interpretation rule into a construction and reporting procedure.

4. Four Families of Checks

The worked example used four checks fitted to one illustrative instrument. Other applications will raise different objections and need different batteries. The available checks fall into four broad families. The first three are null-control families. The fourth varies analyst choices and is therefore a sensitivity family rather than a null control. Each family is defined by the objection it addresses, how it is built, the comparison it warrants, what failure means, and what it cannot establish.

The [companion casebook](#) supplies one machine-readable lesson for each family, plus known-answer and deliberately failing pipeline fixtures. Its answer key and golden-value script make every reported instructional number reproducible. These files demonstrate how a battery is documented; because they are synthetic fixtures, they are not evidence from an enacted research application.

Family or check	Objection addressed	Warranted comparison	Main blind spot
Distributional control	unrelated noise could resemble the pattern	the reference derived or simulated for that construction and statistic; zero only when warranted	may bypass the live application and its data path
Label permutation	the observed difference could reflect label arrangement	its own permutation distribution, when labels could be reassigned without changing the relevant data-generating process	breaks the original grouping–outcome relationship and tests no upstream software
Enacted no-mechanism control	the implemented system could produce the reading without the target mechanism	a matched repeated-control distribution or a derivable known answer	clears only the stated actor–condition and traversed path; it need not model people
Analyst-choice sensitivity	the conclusion could depend on a defensible analytic choice	comparison across prespecified alternative cells, not a null distribution	tests only the choices and branch points that were enumerated

4.1 Distributional controls

Companion worked file: [01_distributional_control.json](#)

Question answered: *Could unrelated noise happen to resemble the pattern being measured?*

How to build it. Generate outcomes from a stated process that lacks the target mechanism, and assign groups independently of those outcomes. For example, generate *Garden Defense* scores from the same score distribution for every actor, randomly assign equal numbers of actors to Red and Blue independently of their scores, and calculate the signed difference between the two group means.

What to compare with. For that construction, the expected signed difference is zero because exchanging Red and Blue reverses the sign without changing the data-generating process. A finite run need not equal 0.0000; repeated runs should fluctuate around zero. This entitlement is specific to the construction and statistic. Ratios, absolute values, maxima, data-dependent exclusions, or groups formed from outcome-related records may require a derived or simulated reference instead.

What failure means. If repeated controls do not behave as the stated construction predicts, inspect the generator, group assignment, filters, and calculation before interpreting the measure.

What it cannot establish. A generated table tests only the pipeline stages it enters. If it bypasses the live application's grouping, logging, or storage, it cannot show what those stages contribute.

4.2 Label permutation

Companion worked file: [02_label_permutation.json](#)

Question answered: *Could this group difference be a lucky arrangement of the labels?*

How to build it. Hold the observed records and calculation fixed, shuffle group labels that are **exchangeable** under the condition being tested, and recompute the same statistic many times. Here, exchangeable means that the labels could be reassigned without changing the relevant process that generated the records. In a *Garden Defense* experiment with Red and Blue assigned independently, keep every score fixed, shuffle the Red and Blue labels while preserving the group sizes, and recalculate the signed gap.

What to compare with. The shuffles produce their own permutation reference. Locate the original group gap within that distribution.

What failure means. A gap far from the shuffled values is difficult to explain as label arrangement alone, under the conditions represented by that shuffle. It is not proof that the proposed human mechanism caused the gap.

What it cannot establish. Shuffling destroys the original grouping–outcome relationship. It therefore cannot reveal an offset created by the same record helping to form a group and determine its outcome, and it does not test upstream application, logging, or storage behavior. Early-High and Early-Low labels derived from Run 1 are not exchangeable with Best Run when Run 1 remains in that outcome. Shuffling those labels is therefore not a warranted reference for the coupled rule in Section 2.3.

4.3 Enacted no-mechanism controls

Companion worked files: `03_enacted_no_mechanism.json`, with known-answer and deliberately failing pipeline fixtures

Question answered: *Could the implemented software and measurement rules produce this reading when the target actor mechanism is absent?*

How to build it. Send a specified actor that lacks the target mechanism through the application. The roll-the-dice actor from Section 2.1 is one example: it makes choices, generates events, reaches storage, encounters filters, receives group labels and outcomes, and enters the same analysis and report intended for people.

What to compare with. Repeat the enacted control and keep its address matched to the question: actor policy, environment, number of actors, observations per actor, exclusions, grouping rule, outcome rule, and statistic. The resulting control distribution—not zero by default—is the warranted reference. When a simple case has a derivable answer, the known answer supplies an additional check, as the mean of 52, 61, and 67 did in Section 2.2.

What failure means. First verify that the actor and pipeline actually ran as specified. If a prespecified or derivable reference exists, an unexpected mismatch requires investigation. If the purpose is to discover an instrument's previously unknown null response, a recurring non-zero pattern is the result to characterize—not automatically a malfunction. In neither case should the control mean be subtracted from the human result without a model that warrants subtraction.

What it cannot establish. Success clears only the named no-mechanism condition and the stages traversed. It does not validate the whole instrument or make the control actor a model of people. Section 2.4's population-adequacy question remains separate.

Online experimentation uses a neighboring practice called *A/A testing*: two arms receive the same experience, so measured differences should follow prespecified chance behavior. Unexpected discrepancies can expose assignment, logging, or carry-over problems (Kohavi et al., 2020). An A/A test and an enacted no-mechanism actor are not identical constructions, but both ask a working system to reveal its behavior under a condition designed not to contain the target difference.

4.4 Analyst-choice sensitivity

Companion worked file: `04_analyst_choice_sensitivity.json`

Question answered: *Does the conclusion depend on a defensible decision made during analysis?*

How to build it. Hold the records fixed and vary prespecified alternatives that could reasonably have been chosen. In *Garden Defense*, the team might compare cutoffs of 60, 65, and 70, examine explicit tie rules, or apply alternative exclusion rules. Best

Run and Last Run should be compared only if both answer defensible versions of the research question; they are not interchangeable outcomes.

What to compare with. The resulting cells are sensitivity results. They are not a null distribution, independent replications, or a menu from which to select the most favorable answer.

What failure means. If a reasonable alternative materially changes the conclusion, report the dependence and resolve the choice from the construct and design—not from which cell looks best.

What it cannot establish. A sweep tests only the choices it enumerates. Write every rule as an executable procedure so that hidden branch points—tie handling, fallback values, filter order, and outcome windows—can be seen. If alternatives are nested or reuse the same records, agreement among them is not independent corroboration.

4.5 Two questions organize the battery

The families are complementary because their blind spots differ. A distributional control may bypass the application. Permutation breaks the very coupling an enacted control may need to preserve. An enacted control does not reveal every analyst choice. A sensitivity sweep removes no target mechanism and misses every choice it fails to enumerate.

The discipline therefore comes down to two questions, asked in order:

- 1 **What objections would a careful reader raise, and which condition or check answers each one?**
- 2 **For each condition, what comparison is warranted by the way it was built?**

The first question follows Abelson's principled argument. The second prevents a control built for one objection from being asked to answer another.

5. Building and Reporting a Battery

The procedure is general even though the battery it produces is fitted to a particular claim, application, measurement rule, and data path.

- 1 **State the actor-level claim.** Write, in one sentence, what the software's records are being used to assert about its users. If that sentence cannot be written, the battery has nothing definite to test.
- 2 **Trace the action-to-report path.** Include the application event, logging, transfer, storage, filtering, grouping, outcome construction, calculation, and report. Each stage is a place where structure can enter.
- 3 **List plausible alternative explanations.** Ask what could produce the reported pattern without the actor mechanism being claimed. Several objections may be addressed by one control, and one objection may require several checks.
- 4 **Fit a condition or check to each objection.** Record which stages it traverses. A table generated directly in a database may test the calculation while bypassing the application, logging, and transfer stages.
- 5 **Specify the control's address.** Name the actor or population, environment, measurement rules, actor count, and observation or run profile. If the result will help interpret people, prespecify the observable features on which the control population must be adequate—for example, run counts, score range, between-actor spread, or stable response styles—and, where feasible, what would count as adequate similarity. Explain why it is a plausible comparison rather than merely a convenient software input.
- 6 **State the purpose and warranted comparison in advance.** Is the condition verifying behavior against a prespecified or derivable answer, or characterizing a previously unknown null response? Identify the expected behavior or reference distribution produced by the construction. For verification, also state what mismatch will trigger investigation. Use a known answer when one can be derived, but do not assume that every no-mechanism condition is entitled to zero.
- 7 **Run and verify the condition.** Record the build, source revision, dependencies, configuration, test fixtures, random seed when relevant, records created, exclusions made, path executed, and settings actually completed. For AI-assisted software, prompt history may supplement this provenance; it cannot replace the code and settings that actually ran.

- 8 **Interpret the result by the kind of check.** For a null control, locate the unchanged statistic within its warranted reference; do not subtract the control mean as an automatic correction. For analyst-choice sensitivity, compare the conclusions across the prespecified alternatives. In both cases, report what the check addressed, what it did not traverse, and what conclusion it can support.
- 9 **Keep a battery register.** For each entry, record the claim, objection, condition or check, represented actor or population, pipeline stages traversed, executed configuration, warranted comparison or reference, result, blind spot, bounded conclusion, and action to take if the check fails.

The register is the battery's visible form. It lets a developer inspect execution, a domain expert inspect the actor and environment, and a researcher inspect the comparison and conclusion. A long simulation report without this map can still hide which objection each result was meant to answer.

5.1 The battery register

A practical register can use one entry per condition or check:

- **Claim:** What is the software's record being used to say about the actor?
- **Objection:** What could produce that reading without the claimed mechanism?
- **Condition or check:** What was removed, broken, held fixed, or varied?
- **Purpose:** Is this entry verifying a known expectation or characterizing an unknown null response?
- **Control address:** Which actor or population, environment, rules, actor count, and observation profile were used?
- **Comparator-fit observables:** If the control will support a claim about people, which observable features must it reproduce, and what would count as adequate similarity?
- **Stages traversed:** Which parts of the action-to-report path did the check actually enter?
- **Executed configuration:** Which build, records, exclusions, settings, iterations, and seed actually ran?
- **Warranted comparison:** What result or reference follows from the construction?
- **Investigation trigger:** For a verification check, what prespecified mismatch requires investigation?
- **Observed result:** What did the check return?
- **Blind spot:** What could this check not test?
- **Bounded conclusion:** Which objection was or was not addressed?
- **Failure action:** What will be investigated, repaired, rerun, added, limited, or withdrawn?

The first entry for *Garden Defense*, for example, would name the claim that later performance indicates learning; the objection that the implemented rules can produce a learning-like reading without learning; the memoryless roll-the-dice actors; all stages from game choice through report; the matched three-run profile; and the repeated enacted-control distribution as the warranted comparison. The result field would contain what the control actually returned, not a presumed zero.

5.2 When a check fails

A checkpoint failure or a verified mismatch with a prespecified or derivable reference must be localized and investigated before the human result is interpreted. A failed check should lead to a stated action rather than a favorable reinterpretation.

- **A checkpoint or known-answer test fails:** stop interpreting the result, locate the missing or changed record or setting, repair the path, and rerun the check.
- **A verified control misses its warranted reference:** investigate unexplained structure in the application, environment, or measure. Do not subtract the control mean and call the remainder corrected.
- **The control actor is a poor comparison for people:** retain it as a software check, but add a plausible no-mechanism population or limit the human claim.
- **A defensible analyst choice changes the conclusion:** report the dependence and resolve the choice from the construct and design, not from the most favorable result.

- **The control behaves as expected:** clear only the named objection. Do not describe the entire pipeline or construct as validated.

5.3 What the battery adds to ordinary software checks

Unit tests, integration tests, logging checks, quality assurance, and usability tests remain necessary. They ask whether components calculate correctly, systems connect, events arrive, features behave as specified, and people can use the product. A null-control battery adds a different question: **what does this working measuring system report when the target actor mechanism is absent or the relationship needed to produce the claimed pattern has been broken?**

The consequences can extend beyond the original measurement. Sambasivan et al. (2021) documented how data problems compounded into downstream failures in high-stakes AI projects, even though practitioners recognized the importance of data quality. They also reported substantial gaps between participants' formal preparation and the practical demands of collecting and working with data. Their study concerns AI development rather than serious-game developers generally, but it shows why awareness of data quality and preparation to examine it cannot be assumed to be the same. For research software, this reinforces the need to examine how records are produced and what later analyses are allowed to infer from them.

A/A tests and known-answer cases are neighboring practices that can strengthen the battery. The battery does not replace them, and it should not be presented as proof that ordinary software testing was inadequate. Its contribution is to connect software execution to the particular inference being made from the software's records. That check matters because a pipeline can return consistent, publishable findings that are unsupported as claims about users. It still does not establish whether the product helps anyone; that is a different question requiring a design fitted to it.

6. Limits and Responsibility

When software shapes an activity and records it, the trace reflects both the actor and the environment that produced it. Before attributing a reported pattern to learning, performance, expertise, or another actor property, the researcher must determine what the environment and measuring procedure produce under a stated no-mechanism condition. A null-control battery is one way to answer that question. When a control also has a derivable expected result, that known answer strengthens the check; it does not replace the battery that locates what was tested and what was not.

A final limit deserves stating rather than assuming. Evidence that an instrument behaved as expected does not make an inference permissible. A well-zeroed measure may provide a better-characterized basis for decisions about individuals, but not a more legitimate one. Consent, purpose limitation, and the question of who may be measured come first, and they are not this report's to settle.

Software that measures people is built by teams whose expertise is software, deployed by institutions whose expertise is the domain, and read by researchers whose expertise is inference. Not one of those three is automatically well placed to notice when the instrumented environment itself has quietly contributed to the finding. AI assistance does not transfer that responsibility away from the developer or researcher who uses the software as an instrument. Somebody has to run the check on purpose, having first asked what the instrument reads when the target mechanism is absent or the required relationship has been broken.

The answer need not be zero. That is the whole of it.

Availability and Status of Instructional Materials

The companion casebook provides the *Garden Defense* and coin examples, synthetic inputs, worked answers, and a script for reproducing the declared calculations. These materials were created for instruction. They contain no human-participant data and are not presented as empirical findings about learning, performance, or expertise. Successful reproduction checks the instructional calculations; it does not validate or certify a real application, construct, or measurement pipeline.

- **Companion casebook:** See [Appendix A](#) for the guided lessons and worked answers.
- **Example data and verification code:** Optional synthetic inputs, calculation-checking code, and instructions are available in the [Praxichnology GitHub repository](#).

Acknowledgments of AI Use

The ideas, research design, analyses, interpretations, and overall composition of this manuscript are the authors' own. During the preparation of this work, the authors used ChatGPT and Claude to assist with drafting and revising the manuscript prose. All AI-assisted text was reviewed, revised, and approved by the authors, who take full responsibility for the content of the published article.

References

- Abelson, R. P. (1995). *Statistics as principled argument*. Lawrence Erlbaum Associates.
- Chiang, H., Liao, C.-J., Wang, Y.-H., Huang, H.-Y., Chen, C.-N., Hsieh, C.-H., Huang, Y.-Y., & Jiang, C.-C. (2010). Comparison of articular cartilage repair by autologous chondrocytes with and without in vitro cultivation. *Tissue Engineering Part C: Methods*, 16(2), 291–300. <https://doi.org/10.1089/ten.tec.2009.0298>
- Dmitriev, P., Gupta, S., Kim, D. W., & Vaz, G. (2017). A dirty dozen: Twelve common metric interpretation pitfalls in online controlled experiments. In *Proceedings of the 23rd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (pp. 1427–1436). Association for Computing Machinery. <https://doi.org/10.1145/3097983.3098024>
- Jacobs, A. Z., & Wallach, H. (2021). Measurement and fairness. In *Proceedings of the 2021 ACM Conference on Fairness, Accountability, and Transparency* (pp. 375–385). Association for Computing Machinery. <https://doi.org/10.1145/3442188.3445901>
- Kohavi, R., Tang, D., Xu, Y., Hemkens, L. G., & Ioannidis, J. P. A. (2020). Online randomized controlled experiments at scale: Lessons and extensions to medicine. *Trials*, 21, Article 150. <https://doi.org/10.1186/s13063-020-4084-y>
- Kovanović, V., Gašević, D., Dawson, S., Joksimović, S., Baker, R. S., & Hatala, M. (2015). Does time-on-task estimation matter? Implications for the validity of learning analytics findings. *Journal of Learning Analytics*, 2(3), 81–110. <https://doi.org/10.18608/jla.2015.23.6>
- Lipsitch, M., Tchetgen Tchetgen, E., & Cohen, T. (2010). Negative controls: A tool for detecting confounding and bias in observational studies. *Epidemiology*, 21(3), 383–388. <https://doi.org/10.1097/EDE.0b013e3181d61eeb>
- Mensor, M. C., Ahlstrom, R. H., & Scheerer, E. W. (1998). Compliant Keeper system replication of the periodontal ligament protective damping function for implants: Part I. *The Journal of Prosthetic Dentistry*, 80(5), 565–569. [https://doi.org/10.1016/S0022-3913\(98\)70033-2](https://doi.org/10.1016/S0022-3913(98)70033-2)
- Sambasivan, N., Kapania, S., Highfill, H., Akrong, D., Paritosh, P., & Aroyo, L. (2021). “Everyone wants to do the model work, not the data work”: Data cascades in high-stakes AI. In *Proceedings of the 2021 CHI Conference on Human Factors in Computing Systems* (Article 39, pp. 1–15). Association for Computing Machinery. <https://doi.org/10.1145/3411764.3445518>

Zeroing the Instrument

Companion casebook: guided lessons and worked answers

Christian S. Loh, Ph.D.

Southern Illinois University

ORCID: 0000-0002-6318-3890

Yanyan Sheng, Ph.D.

The University of Chicago

ORCID: 0000-0003-4871-2202

September 19, 2026

Reading this casebook

Work through each lesson before consulting the answers. Python is optional: the supplied script checks the calculations; it does not generate this book.

Synthetic instructional materials · No human-participant data

A Praxichnology Teaching Resource

github.com/praxichnology/primers

doi.org/10.5281/zenodo.20371003

Appendix A. Companion Casebook

This appendix is the companion casebook to the Primer. It contains the existing guided lessons followed by their worked answers. All examples are synthetic instructional materials. They contain no human-participant data and do not validate or certify a real application, construct, or measurement pipeline.

Read each lesson and attempt its questions before consulting the answer key. The accompanying source package includes the example inputs and the completed battery register. File paths in the lessons are relative to the companion-casebook folder.

Running Python is optional. The command `python3 verify_examples.py` recomputes the example calculations and checks them against the supplied expected results. It prints the numbers and a verification message in the terminal. It does not generate this book or write a new results file.

A.1 Guided lessons

Synthetic instructional example — no human participants, no research data, and no empirical evidence.

Case study box — When “user-generated data” look wrong

A professor and graduate assistant build an online practice environment to study help-seeking and learning. With AI assistance, they add event logging and a dashboard. They think of the resulting records as *user-generated data*: learners answer questions, request hints, and earn scores; the software records what they do.

When the first results appear, something seems off. Learners with more `hint_viewed` events have lower final scores. The team initially wonders whether requesting hints marks weak understanding, or whether the hints themselves are ineffective.

They then inspect the path between learner action and reported finding. The hint panel automatically appears after two incorrect responses, and the software records that display as `hint_viewed` even when the learner never requests or reads the hint. The scoring rule also deducts points for those same incorrect responses. Consequently, two errors both place a learner in the “more hints viewed” group and lower the outcome used to compare that group. Every component may be working exactly as designed, yet the resulting association can appear without help-seeking causing poorer performance—and even without the learner choosing to seek help.

The phrase *user-generated data* was not entirely wrong: learner actions began the process. It was incomplete. The reported pattern was produced through learner responses, the automatic hint trigger, the event definition, the scoring rule, and the later group comparison.

Open the box: Does `hint_viewed` mean that a learner requested help, that the software displayed help, or merely that a logging condition fired? Could an actor that never chooses a hint still enter the “more hints viewed” group? Does the same action help create both the group label and the outcome? What no-mechanism condition would let the team observe what this arrangement reports without learner help-seeking?

How to map the argument

The problem becomes easier to inspect when its parts are named:

- **Claim being considered:** Learners who voluntarily seek more hints show weaker understanding.
- **Observation:** More `hint_viewed` events are associated with lower final scores.
- **Warrant:** `hint_viewed` represents voluntary learner help-seeking, and the software has not created the relationship between that record and the score.

- **Rival explanation:** Two incorrect responses automatically create a `hint_viewed` event and also lower the score, even when the learner never requests help.
- **Fitted check:** Send an actor that makes a specified set of errors but never requests a hint through the implemented application. Check which hint events are recorded, how the actor is grouped, and what score reaches the report.
- **Bounded conclusion:** This check can show whether the implemented arrangement produces the association without voluntary help-seeking. It cannot establish whether requesting or receiving hints helps human learners.

The warrant is the easily missed part: it is the reason the observation is allowed to count as evidence for the claim. Opening the software path exposes assumptions inside that warrant that the phrase *user-generated data* had hidden.

The running case

Garden Defense is a fictional lane-based game. Players place defenses while waves of pests approach a garden. Each run produces a score from 0 to 100, and players may complete several runs. A research team wants to use the game's records to compare groups and discuss repeated performance.

The lessons do not decide whether the fictional game works. They show what different checks can reveal about the machinery that turns actions into a reported result.

For every lesson, map the argument before calculating:

- 1 What claim or interpretation is being examined?
- 2 What recorded or computed observation is being offered as evidence?
- 3 What warrant must hold for that observation to support the claim?
- 4 What rival explanation or objection is the check built to address?
- 5 Which stages of the action-to-report path does the check traverse?
- 6 What comparison is warranted by the check, and what would an unexpected result require the team to do?
- 7 What bounded conclusion is permitted, and what remains outside it?

Lesson 0 — Establish a known answer

File: `data/00_two_toss_known_answer.json`

The report's two-toss example contains four equally likely sequences. Toss 1 creates the group label; Best Toss is the maximum of Toss 1 and Toss 2.

Before calculating, predict whether the First-Head and First-Tail groups should have the same expected Best Toss. Then compute each group mean and the signed difference.

Learning objective: Show that a fair, independent process can produce a non-zero null response when the assigning observation remains inside the outcome.

Lesson 1 — Distributional control

File: `data/01_distributional_control.json`

Both groups draw from the same illustrative score process, and Red/Blue labels are assigned independently of score. The statistic is the signed difference $\text{mean}(\text{Red}) - \text{mean}(\text{Blue})$.

Calculate the two sample means and their difference. Then distinguish the finite sample result from the expected center warranted by the construction.

Learning objective: Separate “expected to be centered at zero” from “every finite sample must equal zero.”

Inspection question: Which application stages were bypassed when the table was supplied directly to the analysis?

Lesson 2 — Label permutation

File: data/02_label_permutation.json

Eight fixed outcomes carry four A and four B labels. The labels were assigned independently, and both groups received the same illustrative experience. Enumerate every assignment of four A labels among the eight outcomes while preserving group sizes.

Compute the observed signed gap, the exact two-sided permutation probability, and the action the team should take.

Learning objective: Use permutation only when the labels are exchangeable under the condition being tested.

Counterexample: Early-High and Early-Low labels derived from Run 1 are not exchangeable with Best Run when Run 1 remains eligible to determine Best Run. Shuffling those labels would destroy the relationship the control needs to preserve.

Lesson 3 — Enacted no-mechanism control

Primary fixture: data/03_enacted_no_mechanism.json **Pipeline fixtures:** data/03a_pipeline_known_answer.json and data/03b_pipeline_dropped_record_failure.json

Eight memoryless actors each complete three runs. Apply the rule Early-High when $\text{Run } 1 \geq 65$; classify the remaining actors as Early-Low. First calculate Best Run across all three runs, including Run 1. Then recompute the gap using later-only Best Run and Last Run.

Every primary-fixture record carries checkpoints for logging, storage, retention, analysis, and reporting. The two small pipeline fixtures contain the same three completed scores. One retains all three; the deliberately failing fixture drops the score of 67 before analysis. Compare each retained count and mean with the derivable answer.

Learning objectives: Identify the built-in effect of reusing Run 1, and distinguish calculation verification from enactment through a real application.

Scope boundary: A loaded JSON fixture is not itself an enacted control. It becomes an integration check only when the application processes it through the declared path. A genuine enacted control requires the application to run the actor and export the resulting traces. A memoryless random actor can test that traversed path without becoming a model of people.

Lesson 4 — Analyst-choice sensitivity

File: data/04_analyst_choice_sensitivity.json

Hold all actor records fixed. Apply the three prespecified Run 1 cutoffs—60, 65, and 70—with ties assigned to Early-High. Calculate the Early-High minus Early-Low Best Run gap for each cell.

Compare the signs and magnitudes. Do not calculate a null probability from these three cells and do not choose the cutoff that gives the preferred conclusion.

Learning objective: Recognize conclusion dependence on an analyst choice without mislabelling the sweep as a null distribution or independent replication.

Closing exercise — Build the battery register

Complete one register entry for each of the four families before reading battery-register.json. A strong entry names the claim, objection, condition, purpose, control address where applicable, stages traversed, executed configuration, warranted comparison, observed result, blind spot, bounded conclusion, and failure action.

The four lessons do not form a universal battery. They are examples of checks fitted to this fictional claim and data path. The first three are null-control families; the fourth is a sensitivity family.

A.2 Worked answers

These answers reproduce synthetic instructional examples. They are not research findings or empirical evidence.

Lesson 0 — Known-answer demonstration

First-Head sequences are HH and HT. Both have Best Toss = 1, so the expected Best Toss is **1.000**. First-Tail sequences are TH and TT, with Best Toss values 1 and 0, so the expected value is **0.500**. The expected signed difference is therefore **0.500**.

Bounded conclusion: The positive response is produced by the rules: Toss 1 creates the labels and remains eligible to determine Best Toss. The coin is fair, the tosses are independent, and no learning is possible.

Lesson 1 — Distributional control

The Red mean is **64.900** and the Blue mean is **64.600**, giving an observed signed difference of **0.300**.

The construction warrants an expected center of zero because group assignment is independent of scores and exchanging the group names reverses the sign without changing the score process. It does not require this finite sample to return exactly 0.000.

Status: Behaved as specified.

Bounded conclusion: The observed 0.300-point gap is a finite realization near the construction's zero center. This generated table does not test the game's logging, storage, filtering, or group-construction code.

Lesson 2 — Label permutation

The observed A mean is **76.750** and the B mean is **65.750**, producing a signed difference of **11.000**. There are $8 \text{ choose } 4 = 70$ assignments that preserve the four-versus-four group sizes. Two assignments produce an absolute gap at least as large as 11.000, so the exact two-sided probability is $2/70 = 0.02857142857142857$.

Status: Requires investigation.

Bounded conclusion: Under the stated exchangeability condition, an 11-point absolute gap is unusual as label arrangement alone. That does not identify an actor mechanism, establish causality, or test upstream application behavior.

Lesson 3 — Enacted no-mechanism control

Using Best Run across all three runs, the Early-High values are 72, 68, 70, and 66, with a mean of **69.000**. The Early-Low values are 58, 62, 64, and 61, with a mean of **61.250**. The signed gap is **7.750**.

The contrast isolates the rule coupling. Later-only Best Run has a mean of **60.750** in both groups, and Last Run has a mean of **59.750** in both groups. Both signed gaps are **0.000**. All **24** records reach every declared checkpoint.

The complete-record pipeline fixture creates and retains three scores—52, 61, and 67—and reproduces their mean of **60.000**. The deliberately failing fixture creates three records but retains only two, changing the analyzed mean to **56.500**. Its one-record shortfall requires investigation.

Status: Characterized for the primary fixture; behaved as specified for 03a; deliberately requires investigation for 03b.

Bounded conclusion: The 7.750-point reading is built into reusing Run 1 for grouping and Best Run in this fixture. It is not a universal null expectation and not an amount to subtract. The supporting pipeline files have derivable answers, but only processing them through a declared application path can test that path. A genuine enacted condition must run the actor through the application.

Lesson 4 — Analyst-choice sensitivity

Run 1 cutoff	Early-High n	Early-Low n	Early-High mean Best	Early-Low mean Best	Signed gap
60	6	4	76.333	71.000	+5.333
65	4	6	73.500	74.667	-1.167
70	2	8	72.500	74.625	-2.125

Status: Requires design-based resolution.

Bounded conclusion: The sign of the comparison depends on the prespecified cutoff. The cutoff must be resolved from the construct and design, not chosen from the most favorable cell. These three cells are neither a null distribution nor three independent replications.